

Programowanie dynamiczne

Najdłuższy wspólny podciąg i optymalne nawiasowanie ciągu macierzy

1 Programowanie dynamiczne

Algorytm programowania dynamicznego powinien wykorzystywać dwie właściwości problemu:

1. **Własność optymalnej podstruktury.** Optymalne rozwiązanie problemu jest funkcją optymalnych rozwiązań podproblemów. Innymi słowy, rozwiązanie całego problemu można skonstruować z optymalnych rozwiązań mniejszych podproblemów.
2. **Własność wspólnych podproblemów.** Prosty algorytm rekurencyjny wielokrotnie rozwiązuje te same podproblemy. Liczba różnych podproblemów powinna być mała, zazwyczaj wielomianowa względem rozmiaru danych wejściowych.

W klasycznej wersji programowania dynamicznego rozwiązanie budujemy *od dołu do góry*: najpierw rozwiązujemy najmniejsze podproblemy, a następnie coraz większe. Alternatywą jest **spamiętywanie** (memoization): rozwiązujemy problem rekurencyjnie od góry do dołu, ale zapisujemy obliczone wyniki, aby nie rozwiązywać tego samego podproblemu wielokrotnie.

2 Problem najdłuższego wspólnego podciągu

2.1 Definicje

Niech

$$X = (x_1, x_2, \dots, x_m)$$

będzie ciągiem.

Definicja 1. Ciąg

$$Z = (z_1, z_2, \dots, z_k)$$

jest **podciągiem** ciągu X , jeśli istnieją indeksy

$$1 \leq i_1 < i_2 < \dots < i_k \leq m$$

takie, że

$$z_j = x_{i_j} \quad \text{dla każdego } j \in \{1, \dots, k\}.$$

Przykład 1. Dla

$$X = (A, B, C, B, D, A, B)$$

ciąg

$$Z = (B, C, D, B)$$

jest podciągiem X , ponieważ można wybrać indeksy $(2, 3, 5, 7)$.

Dane są dwa ciągi

$$X = (x_1, \dots, x_m), \quad Y = (y_1, \dots, y_n).$$

Celem jest znalezienie ich **najdłuższego wspólnego podciągu** (LCS), czyli najdłuższego ciągu będącego podciągiem zarówno X , jak i Y .

Ciąg długości m ma 2^m podciągów, ponieważ dla każdego elementu niezależnie wybieramy, czy należy do podciągu. Algorytm sprawdzający wszystkie podciągi ma zatem czas wykładniczy.

Dla $i \in \{0, \dots, m\}$ oznaczmy

$$X_i = (x_1, \dots, x_i),$$

przy czym $X_0 = \varepsilon$ jest ciągiem pustym. Analogicznie definiujemy Y_j .

2.2 Własność optymalnej podstruktury

Twierdzenie 1 (Struktura optymalnego rozwiązania LCS). *Niech $Z = (z_1, \dots, z_k)$ będzie najdłuższym wspólnym podciągiem ciągów $X = (x_1, \dots, x_m)$ oraz $Y = (y_1, \dots, y_n)$.*

1. *Jeśli $x_m = y_n$, to $z_k = x_m = y_n$, a Z_{k-1} jest najdłuższym wspólnym podciągiem X_{m-1} i Y_{n-1} .*
2. *Jeśli $x_m \neq y_n$, to:*
 - (a) *jeśli $z_k \neq x_m$, to Z jest LCS ciągów X_{m-1} i Y ;*
 - (b) *jeśli $z_k \neq y_n$, to Z jest LCS ciągów X i Y_{n-1} .*

Dowód. 1. Załóżmy, że $x_m = y_n$. Gdyby $z_k \neq x_m$, to wszystkie elementy Z można byłoby wybrać przed ostatnimi pozycjami obu ciągów. Wtedy

$$(z_1, \dots, z_k, x_m)$$

byłby wspólnym podciągiem X i Y długości $k + 1$, co przeczy maksymalności Z . Zatem $z_k = x_m = y_n$.

Ciąg Z_{k-1} jest wspólnym podciągiem X_{m-1} i Y_{n-1} . Gdyby istniał ich wspólny podciąg W o długości większej niż $k - 1$, to przez dopisanie $x_m = y_n$ otrzymalibyśmy wspólny podciąg X i Y dłuższy niż Z . Jest to sprzeczność.

2. Załóżmy, że $x_m \neq y_n$. Ostatni element Z nie może być jednocześnie równy x_m i y_n , więc zachodzi co najmniej jeden z warunków $z_k \neq x_m$ lub $z_k \neq y_n$.

Jeśli $z_k \neq x_m$, to Z jest podciągiem X_{m-1} oraz Y . Gdyby istniał dłuższy wspólny podciąg tych dwóch ciągów, byłby on również wspólnym podciągiem X i Y , co przeczyłoby optymalności Z . Drugi przypadek jest analogiczny. $\square$

2.3 Zależność rekurencyjna

Niech $c[i, j]$ oznacza długość najdłuższego wspólnego podciągu X_i i Y_j . Z poprzedniego twierdzenia otrzymujemy

$$c[i, j] = \begin{cases} 0, & i = 0 \text{ lub } j = 0, \\ c[i - 1, j - 1] + 1, & i, j > 0 \text{ oraz } x_i = y_j, \\ \max\{c[i - 1, j], c[i, j - 1]\}, & i, j > 0 \text{ oraz } x_i \neq y_j. \end{cases}$$

Oprócz tablicy c przechowujemy tablicę ℓ opisującą, z którego podproblemu pochodzi rozwiązanie:

$$\ell[i, j] \in \{\nwarrow, \uparrow, \leftarrow\}.$$

2.4 Algorytm LCS

Algorytm 1 LCS(X, Y)

```

1:  $m \leftarrow \text{Length}(X)$ 
2:  $n \leftarrow \text{Length}(Y)$ 
3: for  $i = 0$  to  $m$  do
4:    $c[i, 0] \leftarrow 0$ 
5: end for
6: for  $j = 0$  to  $n$  do
7:    $c[0, j] \leftarrow 0$ 
8: end for
9: for  $i = 1$  to  $m$  do
10:  for  $j = 1$  to  $n$  do
11:    if  $x_i = y_j$  then
12:       $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ 
13:       $\ell[i, j] \leftarrow \nwarrow$ 
14:    else if  $c[i - 1, j] \geq c[i, j - 1]$  then
15:       $c[i, j] \leftarrow c[i - 1, j]$ 
16:       $\ell[i, j] \leftarrow \uparrow$ 
17:    else
18:       $c[i, j] \leftarrow c[i, j - 1]$ 
19:       $\ell[i, j] \leftarrow \leftarrow$ 
20:    end if
21:  end for
22: end for
23: return  $(c, \ell)$ 

```

Tablice mają odpowiednio $(m + 1)(n + 1)$ oraz mn elementów. Każdy element obliczamy w czasie stałym, dlatego

$$T(m, n) = \Theta(mn), \quad S(m, n) = \Theta(mn).$$

2.5 Odtwarzanie najdłuższego wspólnego podciągu

Algorytm 2 Print-LCS(ℓ, X, i, j)

```

1: if  $i = 0$  or  $j = 0$  then
2:   return
3: end if
4: if  $\ell[i, j] = \nwarrow$  then
5:   Print-LCS( $\ell, X, i - 1, j - 1$ )
6:   print  $x_i$ 
7: else if  $\ell[i, j] = \uparrow$  then
8:   Print-LCS( $\ell, X, i - 1, j$ )
9: else
10:  Print-LCS( $\ell, X, i, j - 1$ )
11: end if

```

W każdym wywołaniu zmniejsza się co najmniej jeden z indeksów, dlatego czas działania i maksymalna głębokość rekurencji wynoszą $O(m + n)$.

2.6 Przykład

Dla

$$X = (A, B, C, B, D, A, B), \quad Y = (B, D, C, A, B, A)$$

algorytm tworzy następującą tablicę długości:

	0	1	2	3	4	5	6
	ε	B	D	C	A	B	A
ε	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

Tablica kierunków ma postać:

	1	2	3	4	5	6
	B	D	C	A	B	A
A	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\leftarrow$	$\nwarrow$
B	$\nwarrow$	$\leftarrow$	$\leftarrow$	$\uparrow$	$\nwarrow$	$\leftarrow$
C	$\uparrow$	$\uparrow$	$\nwarrow$	$\leftarrow$	$\uparrow$	$\uparrow$
B	$\nwarrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\leftarrow$
D	$\uparrow$	$\nwarrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$
A	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\uparrow$	$\nwarrow$
B	$\nwarrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\uparrow$

Przy przyjętej regule rozstrzygania remisów ($\uparrow$) procedura PRINT-LCS zwraca

$$(B, C, B, A).$$

Długość LCS wynosi $c[7, 6] = 4$. Inne najdłuższe wspólne podciągi mogą również istnieć.

3 Problem optymalnego nawiasowania ciągu macierzy

3.1 Definicja problemu

Niech

$$p = (p_0, p_1, \dots, p_n)$$

będzie ciągiem wymiarów, a A_i macierzą o wymiarach

$$p_{i-1} \times p_i, \quad i \in \{1, \dots, n\}.$$

Jeśli macierz A ma wymiary $p \times q$, a macierz B ma wymiary $q \times r$, to standardowe obliczenie iloczynu AB wymaga

$$pqr$$

mnożeń skalnych.

Mnożenie macierzy jest łączne, więc wynik iloczynu

$$A_1 A_2 \cdots A_n$$

nie zależy od nawiasowania. Koszt obliczeń może jednak bardzo silnie zależeć od kolejności wykonywania mnożeń.

Problem Matrix-Chain-Order: znaleźć takie pełne nawiasowanie ciągu macierzy, dla którego liczba mnożeń skalnych jest najmniejsza.

3.2 Własność optymalnej podstruktury

Rozważmy podciąg

$$A_i A_{i+1} \cdots A_j.$$

Każde pełne nawiasowanie wykonuje ostatnie mnożenie dla pewnego k , $i \leq k < j$:

$$(A_i \cdots A_k)(A_{k+1} \cdots A_j).$$

Twierdzenie 2. *Jeżeli optymalne nawiasowanie ciągu $A_i \cdots A_j$ wykonuje ostatnie mnożenie między A_k i A_{k+1} , to nawiasowania podciągów $A_i \cdots A_k$ oraz $A_{k+1} \cdots A_j$ są optymalne.*

Dowód. Przypuśćmy, że nawiasowanie $A_i \cdots A_k$ nie jest optymalne. Można je wtedy zastąpić nawiasowaniem o mniejszym koszcie, pozostawiając bez zmian prawą część i ostatnie mnożenie. Otrzymalibyśmy nawiasowanie $A_i \cdots A_j$ tańsze od rozwiązania optymalnego, co jest sprzecznością. Analogiczny argument stosujemy do prawego podciągu. $\square$

3.3 Zależność rekurencyjna

Niech $m[i, j]$ oznacza minimalną liczbę mnożeń skalnych potrzebnych do obliczenia $A_i \cdots A_j$. Dla pojedynczej macierzy nie wykonujemy mnożeń:

$$m[i, i] = 0.$$

Jeżeli ostatnie mnożenie wykonujemy po podziale w pozycji k , to:

- obliczenie $A_i \cdots A_k$ kosztuje $m[i, k]$,

- obliczenie $A_{k+1} \cdots A_j$ kosztuje $m[k+1, j]$,
- wynik lewej części ma wymiary $p_{i-1} \times p_k$,
- wynik prawej części ma wymiary $p_k \times p_j$,
- ich pomnożenie kosztuje $p_{i-1}p_kp_j$.

Zatem

$$m[i, j] = \begin{cases} 0, & i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}, & i < j. \end{cases}$$

W tablicy $s[i, j]$ zapisujemy indeks k , dla którego uzyskano minimum.

3.4 Algorytm od dołu do góry

Algorytm 3 Matrix-Chain-Order(p)

```

1:  $n \leftarrow \text{Length}(p) - 1$ 
2: for  $i = 1$  to  $n$  do
3:    $m[i, i] \leftarrow 0$ 
4: end for
5: for  $l = 2$  to  $n$  do ▷ długość podciągu macierzy
6:   for  $i = 1$  to  $n - l + 1$  do
7:      $j \leftarrow i + l - 1$ 
8:      $m[i, j] \leftarrow \infty$ 
9:     for  $k = i$  to  $j - 1$  do
10:       $q \leftarrow m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$ 
11:      if  $q < m[i, j]$  then
12:         $m[i, j] \leftarrow q$ 
13:         $s[i, j] \leftarrow k$ 
14:      end if
15:    end for
16:  end for
17: end for
18: return  $(m, s)$ 

```

Istnieje $\Theta(n^2)$ podproblemów (i, j) . Dla każdego sprawdzamy co najwyżej $n - 1$ możliwych miejsc podziału, dlatego

$$T(n) = \Theta(n^3), \quad S(n) = \Theta(n^2).$$

3.5 Odtwarzanie optymalnego nawiasowania

Algorytm 4 Print-Optimal-Parens(s, i, j)

```
1: if  $i = j$  then
2:   print  $A_i$ 
3: else
4:   print "("
5:   Print-Optimal-Parens( $s, i, s[i, j]$ )
6:   Print-Optimal-Parens( $s, s[i, j] + 1, j$ )
7:   print ")"
8: end if
```

3.6 Wersja ze spamiętywaniem

Wersja ze spamiętywaniem rozpoczyna od problemu $(1, n)$ i oblicza tylko te podproblemy, których wyniki są potrzebne. Każda obliczona wartość zostaje zapisana w tablicy m .

Algorytm 5 Memoized-Matrix-Chain(p)

```
1:  $n \leftarrow \text{Length}(p) - 1$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = i$  to  $n$  do
4:      $m[i, j] \leftarrow \infty$ 
5:   end for
6: end for
7: return Lookup-Chain( $p, 1, n$ )
```

Algorytm 6 Lookup-Chain(p, i, j)

```
1: if  $m[i, j] < \infty$  then
2:   return ( $m[i, j], s[i, j]$ )
3: end if
4: if  $i = j$  then
5:    $m[i, j] \leftarrow 0$ 
6: else
7:   for  $k = i$  to  $j - 1$  do
8:      $q_1 \leftarrow \text{Lookup-Chain}(p, i, k)$ 
9:      $q_2 \leftarrow \text{Lookup-Chain}(p, k + 1, j)$ 
10:     $q \leftarrow q_1 + q_2 + p_{i-1}p_kp_j$ 
11:    if  $q < m[i, j]$  then
12:       $m[i, j] \leftarrow q$ 
13:       $s[i, j] \leftarrow k$ 
14:    end if
15:   end for
16: end if
17: return ( $m[i, j], s[i, j]$ )
```

Tablica zawiera $\Theta(n^2)$ możliwych podproblemów. Wartość każdego podproblemu jest obliczana tylko raz, a jej obliczenie wymaga sprawdzenia $O(n)$ podziałów. Ostatecznie

$$T(n) = \Theta(n^3), \quad S(n) = \Theta(n^2).$$